

420-301-VA: Web Services

Lab 4: Advanced API Design, OpenAPI, and Project Setup

Pravish Sainath
Computer Science & Technology
Vanier College

Introduction

Up to this point, we have explored the raw mechanisms of HTTP and sockets. We are now transitioning into the **Software Engineering** and **System Design** phase of building modern web services. Before writing a single line of backend Java code, a professional team must establish a clear contract of how their API will look and behave, and set up a structured, professional collaborative environment.

This lab focuses on “Contract-First” development. You will learn how to design clean, RESTful URIs that scale, document them using the industry-standard **OpenAPI Specification (OAS)** (including complex request bodies and path parameters), and configure your GitHub repository to manage your team’s workflow using Agile Kanban boards, Milestones, and professional documentation.

Objectives

By the end of this lab, you should be able to:

- Apply RESTful best practices to design resource-oriented URIs, avoiding common anti-patterns.
- Utilize query parameters for pagination, sorting, and filtering.
- Write a comprehensive OpenAPI 3.0 specification in YAML to document your API’s endpoints, methods, parameters, and reusable data schemas.
- Set up a professional GitHub repository with a structured `README.md` and branch protection rules.
- Configure a GitHub Project board (Kanban) and Milestones to track issues and team assignments.

Instructions

- This lab contains two tracks: an **Illustrative Example** (A Library Management System) containing step-by-step guided exercises to help you practice, and the **Team Project Application** where you apply these exact same skills to your actual course project.
- For the OpenAPI exercises, use the web-based [Swagger Editor](#) to get live feedback, syntax highlighting, and error checking on your YAML syntax.
- Work with your project team members for the repository setup portions, but ensure everyone actively participates in the API design decisions.

1 Part 1: Advanced Resource and URI Design

1.1 Concept Review: RESTful Best Practices and Anti-Patterns

In REST (Representational State Transfer), the URL should only describe the **Resource** (a noun), while the HTTP Method (GET, POST, PUT, DELETE) describes the **Action** (the verb).

Here are the golden rules for designing URIs, and the anti-patterns to avoid:

1. Use Nouns, Not Verbs:

- **Bad:** /getAllUsers or /createUser
- **Good:** /users (The verb is implied by the HTTP method: GET or POST).

2. Use Plurals Consistently:

- **Bad:** /user/123
- **Good:** /users/123. (Read this as: “Fetch ID 123 from the collection of users”).

3. Show Hierarchy (Nesting): If a resource belongs to another resource, nest it.

- **Good:** /users/123/orders (Retrieves the orders belonging to user 123).
- *Limit nesting to one or two levels.* /users/123/orders/456/items/789 becomes too complex.

4. Use Query Parameters for Filtering, Sorting, and Pagination: Do not create new paths for specific views of data.

- **Bad:** /books/author/tolkien or /books/available
- **Good:** /books?author=tolkien&status=available&page=2&sort=year

Exercise 1: Designing Endpoints

Step 1A (Practice): Fixing Bad URIs

Analyze the following “bad” API endpoints. Create a text file and write down the corrected, RESTful version for each, along with the appropriate HTTP method.

1. GET /updateBookPrice?id=45&price=9.99
2. POST /deleteUser/102
3. GET /getAvailableCarsInMontreal
4. POST /addNewCommentToPost?postId=55

Step 1B (Practice): The Library System Domain Modeling

Imagine you are building a Library Management System. Identify the core resources and write down the URIs and HTTP methods needed to perform the following actions:

- Retrieve a paginated list of all books (e.g., page 2, 20 items per page).
- Add a completely new book to the library inventory.

- Retrieve the details of a specific book with ID 42.
- Completely replace the details of book 42.
- Delete book 42 from the system.
- Retrieve a list of all reviews written specifically for book 42 (Hierarchical relationship).
- Post a new review for book 42.

Step 1C (Your Team Project):

Sit down with your team. Identify the top 3 to 5 core resources for your specific project (e.g., if you are building an e-commerce site, your resources might be **Products**, **Users**, **Orders**, and **Categories**).

- Draft the URIs, Query Parameters, and corresponding HTTP methods for the CRUD (Create, Read, Update, Delete) operations of these primary resources.
- Ensure you discuss filtering (e.g., searching by name, filtering by date).
- Save this draft; you will need it for Part 2.

2 Part 2: The OpenAPI Specification (OAS)

2.1 Concept Review: Documenting the Contract

The OpenAPI Specification (formerly Swagger) is a standard, language-agnostic interface to RESTful APIs. It allows both humans and computers to discover and understand the capabilities of your service without looking at source code. We write it in YAML format.

An OAS document has several key sections:

- **info:** Metadata about the API (title, version).
- **servers:** The base URLs where the API is hosted (e.g., localhost for dev, a real URL for production).
- **paths:** The endpoints (URIs) and the HTTP methods they support.
- **components:** Reusable data models (schemas), security schemes, and responses. Using `$ref` allows you to define a `Book` schema once and reuse it across multiple endpoints.

```

openapi: 3.0.3
info:
  title: Library API
  version: 1.0.0
  description: A comprehensive API to manage library inventory and patrons.
servers:
  - url: http://localhost:8080/api/v1
    description: Local Development Server
paths:
  /books:
    get:
      summary: Get a list of all books
      description: Retrieves a list of books. Can be filtered by availability.
      responses:
        '200':
          description: A JSON array of books
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Book'
components:
  schemas:
    Book:
      type: object
      properties:
        id:
          type: integer
          example: 42
        title:
          type: string
          example: "The Hobbit"
        isAvailable:
          type: boolean
          example: true

```

Listing 1: Example: A basic OpenAPI 3.0 YAML skeleton for a Book resource

Exercise 2: Writing the Spec

Step 2A (Practice): Swagger Editor Intro

Go to editor.swagger.io. Clear the default text and paste the YAML snippet provided on the previous page. Look at the UI generated on the right side of the screen.

Step 2B (Practice): Adding Path Variables

You need to document how to fetch a single book by its ID. Add a new path for `/books/{id}`. You must define a `parameters` array to tell OpenAPI that `id` is a variable in the path.

Add this below your `/books` path block:

```
/books/{id}:
  get:
    summary: Retrieve a specific book by ID
    parameters:
      - in: path
        name: id
        required: true
        schema:
          type: integer
        description: The unique identifier of the book
    responses:
      '200':
        description: A single Book object
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Book'
      '404':
        description: Book not found
```

Step 2C (Practice): Defining a POST Request (Request Body)

When creating a new resource, the client sends data in the request body. Add a `post` method under the `/books` path.

```
/books:
  get:
    # ... (existing GET code)
  post:
    summary: Add a new book to the library
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Book'
    responses:
      '201':
        description: Book successfully created
```

Step 2D (Your Team Project):

Create a new file on your computer named `openapi.yaml`. Using the URIs and resources you designed in Step 1C, write the initial OpenAPI specification for your team project.

- Define your `info` and `servers` blocks.
- Define your core `components/schemas`. Think carefully about the properties (e.g., strings, integers, arrays) that make up your data models.
- Document at least 5 endpoints (e.g., `GET` all, `GET` by ID, `POST` create, `PUT` update, `DELETE`).
- Include at least one example of a query parameter for filtering or searching.
- Validate your file using the Swagger Editor to ensure there are no indentation or syntax errors. Save this file.

3 Part 3: Professional GitHub Workflow

3.1 Concept Review: Professional Infrastructure

A professional repository is more than just a dump for code. It requires structure, documentation, and safeguards.

- **README.md:** The landing page of your project. It explains what the project is, how to install it, and how to use it. A repository without a good README is essentially useless to outside developers.
- **Branch Protection:** In a team environment, no one should be allowed to push code directly to the `main` branch. Code should be pushed to a feature branch and merged via a Pull Request (PR) after review.
- **Kanban Boards & Milestones:** Agile development relies on tracking units of work (Issues) through stages (To Do → In Progress → Done) and grouping them into sprints or deliverables (Milestones).

Exercise 3: Setting up the Team Hub

Step 3A: Repository Initialization & Branch Protection

- Have one team member create a new GitHub Repository for your project. Invite all other members as collaborators (Settings → Collaborators).
- Go to **Settings** → **Branches**. Click **Add branch protection rule**.
- Set the branch name pattern to `main`.
- Check the box for **Require a pull request before merging**. (Optional but recommended: Require at least 1 approval before merging).
- Save the rule. Now, everyone must create separate branches for their work!

Step 3B: Creating the README

- Create a `README.md` in the root directory. Use Markdown syntax to format it nicely. It **must** include the following sections:
 1. **Project Title & Description:** A compelling paragraph explaining what this web service does and the problem it solves.
 2. **Team Members:** Names and specific roles/responsibilities.
 3. **Tech Stack:** A bulleted list of the technologies you will use (e.g., Java 17, Spring Boot 3, PostgreSQL, Docker).
 4. **API Documentation:** Create a folder named `docs/` in your repository. Commit your `openapi.yaml` file there. Add a link in the README pointing to this file.

Step 3C: Milestones & Issues

- Navigate to the **Issues** tab in your GitHub repository. Click on **Milestones** (next to the search bar) and create a new Milestone. Call it **Sprint 1 - Minimum Viable Product (MVP)** and set a due date for 2 weeks from today.
- Go back to **Issues** and create at least **8 initial Issues** that represent the work needed to build your backend. Examples include:
 - “Initialize Spring Boot project skeleton”
 - “Configure Database Connection in application.properties”
 - “Implement User Entity and Repository”
 - “Create REST Controller for Products”
- Assign every issue to the **Sprint 1** Milestone.

Step 3D: GitHub Projects (The Kanban Board)

- Navigate to the **Projects** tab (at the repository level or organization level) and create a new Project. Select the **Board** template.
- Ensure you have three standard columns: **To Do**, **In Progress**, and **Done**.
- Click “Add Item” at the bottom of the To Do column and search for the issues you just created (typing “ usually brings them up).
- **Action:** Assign at least one issue to each team member. Have each team member log into GitHub, find their assigned issue, and drag it from “To Do” into “In Progress”.

References and Further Reading

API and URI Design

- [Microsoft: RESTful Web API Design Best Practices](#): An excellent, comprehensive guide on how to design logical, resource-oriented URIs, properly utilize HTTP methods, and implement pagination.
- [Zalando RESTful API Guidelines](#): One of the most famous industry standards for API design. Highly recommended for understanding filtering and standardizing JSON responses.

OpenAPI / Swagger

- [Swagger: Basic Structure of OpenAPI](#): The official guide explaining how to write YAML for paths, parameters (query, path, header), and schemas.
- [Swagger: Describing Request Bodies](#): Detailed instructions on defining POST and PUT payloads.
- [Swagger Editor](#): The live browser-based editor for writing and validating your OpenAPI YAML files.
- [The Official OpenAPI 3.0.3 Specification](#): The deep-dive technical documentation for everything supported by the standard.

GitHub and Agile Project Management

- [GitHub Markdown Guide](#): Learn how to format your README.md with headers, bold text, code blocks, and links to make your documentation look professional.
- [GitHub Docs: Branch Protection](#): Understand how branch protection rules enforce code review and prevent accidental deletion of important code.
- [GitHub Docs: About Projects](#): A guide on how to use GitHub Projects to create Kanban boards, automate issue tracking, and collaborate effectively as a team.
- [Atlassian: What is Kanban?](#): A great conceptual overview of why we use To-Do, In-Progress, and Done boards to manage software engineering tasks and limit work-in-progress.